set of [mem] vectors. At inference time, we start generation with the learned [mem] tokens and let LM to decode the text.

Let's estimate an upper bound on the number of tokens that can be generated from a single input vector by a language model. The input vector has a dimension d_{model}, with each element represented by b bits, so that the total information content is approximately $d_{model} \times b$ bits. Given a vocabulary of size $|\mathcal{V}|$, where each token carries at most $\log_2 |\mathcal{V}|$ bits of information, the maximum number of tokens L that can be generated is bounded by:

$$L \leq \frac{d_{model} \times b}{\log_2 |\mathcal{V}|}. \quad (1)$$

Our goal is to quantify the capacity of trainable input vectors (denoted as [mem]) in terms of the amount of information they can encode and later decode. From an information-theoretic standpoint, we interpret this capacity as the ability to reduce uncertainty in the generated text. To this end, we define the following metrics.

Decoding Capacity (in Tokens): From an information-theoretic perspective, this metric represents the maximum number of tokens that can be reliably reconstructed from the compressed representation in the [mem] vectors. It is defined as the longest text sequence length L for which the token-level accuracy exceeds a predefined threshold:

$$L_{\text{max}} = \max \big\{ L \mid \text{Acc}\big(\text{LM}(t_{[1:L]} \mid \texttt{[mem]})\big) > \text{thr} \big\}, \quad (2)$$

here, *Acc* is computed via teacher-forcing when decoding text both with and without the [mem] vectors. This measure reflects the effective storage limit (in tokens) imposed by the fixed capacity of the memory vector.

Token Gain: This metric estimates the additional number of tokens that can be correctly decoded due to the presence of the [mem] vector, relative to the baseline performance of the language model (LM) without it. Formally, if $C_{\text{tokens}}^{\text{LM+[mem]}}$ is the count of tokens correctly predicted when using the memory vector and $C_{\text{tokens}}^{\text{LM}}$ is the count without it, then the gain is given by

$$\begin{aligned} C_{\text{tokens}} &= C_{\text{tokens}}^{\text{LM+[mem]}} - C_{\text{tokens}}^{\text{LM}} \\ &= \sum_{i=1}^{N} \mathbb{1}\Big(t_i = \text{LM}(t_{[1:i-1]} \mid \texttt{[mem]})\Big) \\ &\quad - \sum_{i=1}^{N} \mathbb{1}\Big(t_i = \text{LM}(t_{[1:i-1]})\Big). \end{aligned} \quad (3)$$

Viewed through an information-theoretic lens, this difference quantifies the number of tokens' worth of information (i.e., discrete units) that the memory vector adds to the decoding process.

Information Gain: Cross-entropy measures the uncertainty or the average number of bits required to encode a sequence under a given model. The *Information Gain* quantifies how much the [mem] vector reduces this uncertainty. Let $H_{\text{LM}} = H\big(P_\theta(t_{[1:N]})\big)$ be the cross-entropy (in bits) when decoding without the memory vector, and $H_{\text{LM+[mem]}} = H\big(P_\theta(t_{[1:N]} \mid \texttt{[mem]})\big)$ be the cross-entropy with the memory vector. Then, the reduction is given by

$$\text{CE-reduction} = C_{\text{H}} = H_{\text{LM}} - H_{\text{LM+[mem]}}. \quad (4)$$

This measures how many fewer bits are needed to represent the text, thus reflecting the additional information provided by the memory vector.

Collectively, these metrics enable us to characterize the capacity of the trainable input vectors both in terms of discrete tokens (C_{tokens}) and entropy (C_{H}), while L_{max} provides an upper bound on the length of text that can be accurately reconstructed. In our experiments, these measures are computed over a curated set of texts and averaged to obtain robust estimates. We note that the absolute values of *Information Gain* depend on the underlying vocabulary, and therefore should not be directly compared across models with different vocabularies.

4 Experiments and Results

We evaluate capacity of trainable input vectors of the same size as dimension of input embeddings for different language models on texts from different sources.

Models We use models from Pythia suite (160M, 410M, 1.4B, 2.8B) (Biderman et al., 2023), OPT-1.3B (Zhang et al., 2022), OLMo-1B (Groeneveld et al., 2024), Sheared-LLaMA-1.3B (Xia et al., 2024), Llama-3 models (1B, 3B, 8B) (Dubey et al., 2024), and Mamba (130M, 370M, 790M, 1.4B) (Gu and Dao, 2024). List of all used models with links to HuggingFace Hub are in Appendix A.

Data As a source of texts for compression, we use texts from the PG-19 dataset (Rae et al., 2020), which consists of books extracted from the Project Gutenberg library. Given that PG-19 is publicly available and contains books, it is highly plausible

		Pythia-160M	Pythia-410M	Pythia-1.4B	Llama-3.2-1B	Llama-3.2-3B	Llama-3.1-8B
PG-19	Max, tokens	80	96	160	512	1024	1568
	Gain, tokens	$70.9_{\pm 11.0}$	$81.3_{\pm 12.0}$	$158.0_{\pm 29.1}$	$426.2_{\pm 79.2}$	$720.3_{\pm 80.2}$	$1094.1_{\pm 127.6}$
	Information Gain	$396.4_{\pm 46.0}$	$431.4_{\pm 51.6}$	$792.8_{\pm 143.4}$	$2119.9_{\pm 364.8}$	$3292.2_{\pm 320.0}$	$4865.7_{\pm 546.6}$
Fanfics	Max, tokens	80	96	192	512	1024	1568
	Gain, tokens	$70.9_{\pm 10.5}$	$81.2_{\pm 11.6}$	$152.9_{\pm 28.0}$	$449.6_{\pm 83.7}$	$734.1_{\pm 85.0}$	$1071.8_{\pm 168.6}$
	Information Gain	$378.1_{\pm 45.9}$	$429.8_{\pm 46.2}$	$776.9_{\pm 132.5}$	$2213.8_{\pm 365.8}$	$3354.5_{\pm 344.9}$	$4768.9_{\pm 622.6}$
Random	Max, tokens	65	72	139	316	460	792
	Gain, tokens	$61.3_{\pm 6.6}$	$76.9_{\pm 8.7}$	$144.4_{\pm 17.5}$	$294.9_{\pm 64.8}$	$456.9_{\pm 72.1}$	$623.2_{\pm 97.3}$
	Information Gain	$500.8_{\pm 38.9}$	$630.4_{\pm 65.2}$	$1108.2_{\pm 136.2}$	$2265.2_{\pm 498.7}$	$3382.6_{\pm 585.2}$	$4541.2_{\pm 758.6}$

Table 1: **Compression capacity across different text sources and models.** We report *Decoding Capacity (in Tokens)* ("Max, tokens" in the Table), *Token Gain*, and *Information Gain* for texts from *PG-19*, *fanfics*, *random*. Notably, *Information Gain* remains similar across all text sources for each model (except *random* for Pythia). For *PG-19* and *fanfics*, LMs leverage their ability to predict natural language, so the *Decoding Capacity (in Tokens)* generally exceeds the *Token Gain*. Furthermore, we find no evidence that the models benefit from potentially having PG-19 in their pre-training data, as their performance on *PG-19* is not significantly better than on *fanfics* published after October 2024. In contrast, random text offers no predictable structure, making these two metrics nearly identical. This allows us to distinguish how many tokens model can predict by itself compared to decoding from trainable input vector. Larger models consistently show greater compression capacity across all metrics.

that these texts were included in the pre-training data of LLMs. Notably, PG-19 is part of the Pile dataset (Gao et al., 2020), which was used to train Pythia models.

To assess the compression of texts that models have not encountered during pre-training, we collected fanfiction texts published online after October 2024 from the AO3 fanfics library[2]. Details of this collection process are provided in Appendix B.

Both the *PG-19* and *fanfics* consist of natural language texts, where language models can predict some tokens based on prior context and model parameters. To isolate the capacity of the trainable input vectors without the influence of the knowledge of natural language by language model itself, we also employed *random* texts. *Random* texts were generated by randomly sampling words from the top 100,000 words from the GloVe vocabulary[3].

We train only a set of M vectors that are prepended to the model's input. In most of the experiments, we use only one trainable vector, if not stated otherwise.

4.1 Decoding Capacity of a Single Vector

We find that a *single* trainable vector can enable language models to produce surprisingly long, *targeted* text sequences. We estimate *Decoding Capacity (in Tokens)* (Eq. (2)) on 50 texts from PG-19 for each length. We set a token-level accuracy threshold of 0.99 and evaluate across the following length grid: [64, 80, 96, 128, 160, 192, 256, 384, 512, 768, 1024, 1280, 1568, 2048, 2560, 3072].

Fig. 1 presents the results for the evaluated models. Notably, Llama-3.1-8B can accurately reconstruct texts of up to 1568 tokens from just a *single* input vector. Interestingly, among models with around 1B parameters (Pythia-1.4B, OPT-1.3B, OLMo-1B, Sheared-LLaMA-1.3B, and Llama-3.2-1B) we observe compressive capacity that ranges from 128 to 512 tokens. Pythia-2.8b, despite its larger size, has poor compression of just 128 tokens compared to smaller 1B models.

4.2 Memorization, Natural Language Understanding and Episodic Memory

Generation from the `[mem]` vector involves combining information from both the pre-trained language model parameters and memory about text specific sequence. To analyze contributions of these different types of memory, we use *Token Gain* (Eq. (3)) which measure the extra number of tokens predicted correctly, and *Information Gain* (Eq. (4)) showing the decrease in cross-entropy when decoding from memory vector. In contrast to *Decoding Capacity*, these two metrics more directly isolate the capacity contributed by the `[mem]` vector itself.

In addition to texts from *PG-19* that may have been seen by LMs during pre-training, we consider: (1) texts from *fanfics* to factor out memorization as they were published after release of the models, and

[2] https://archiveofourown.org/
[3] https://nlp.stanford.edu/data/glove.6B.zip

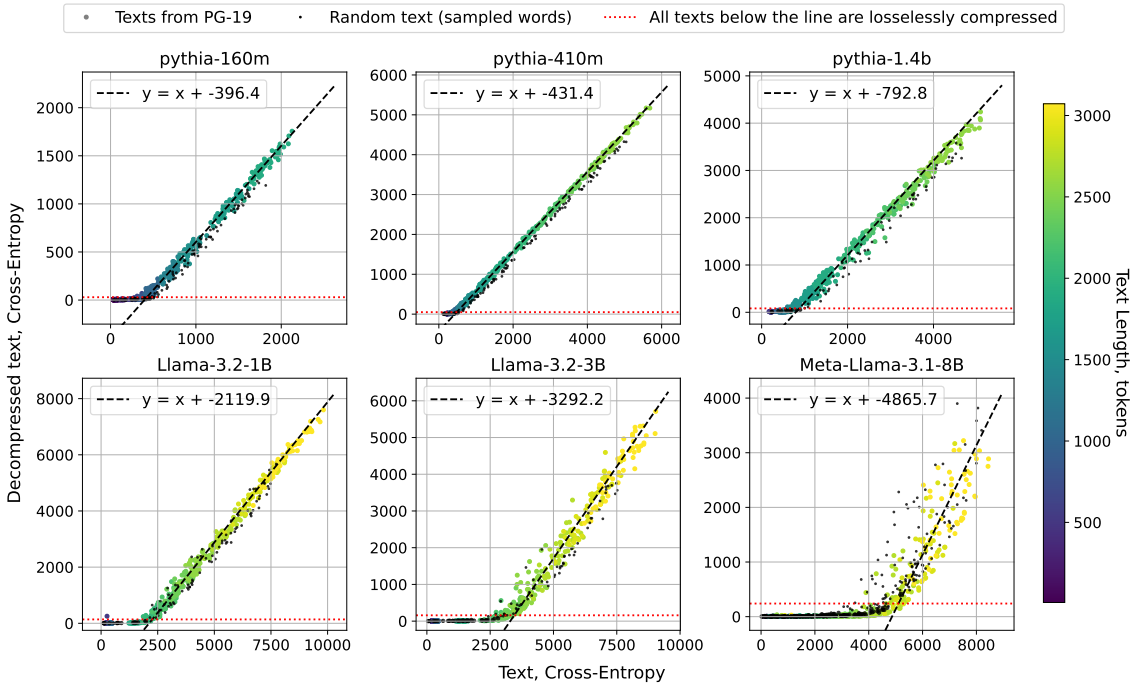

Figure 3: **Information gain of text compression to `[mem]` vector doesn't depend on language understanding capabilities of models.** Compression results for various language models show the relationship between the cross-entropy (CE) of the original and decompressed texts. If the text CE falls below a model-specific threshold (red line), the text is losslessly compressed. This value is a input vector capacity in terms of entropy (*Information Gain*, C_H). For texts that are not perfectly compressed, the compression process reduces their CE to a consistent, model-specific value (bias of the black dashed line). Larger models (e.g., Llama-3.1-8B) can handle longer texts before reaching the compression threshold, due to their greater capacity compared to smaller models (e.g., Pythia-160M). This behavior holds for both natural texts (*PG-19*) and unnatural *random* texts consisting of random word sequences.

(2) *random* sequenses of words to exclude learned natural language understanding capabilities.

Decoding Capacity (in Tokens) for texts from *PG-19* and *fanfics* was evaluated on the following length grid: [64, 80, 96, 128, 160, 192, 256, 384, 512, 768, 1024, 1280, 1568, 2048, 2560, 3072].

Table 1 summarizes the results for each model and text source. We have two main observations. The metrics for both *PG-19* and *fanfics* are remarkably similar across all models tested. This similarity implies that the presence of *PG-19* in the pre-training data does not provide much of an advantage. Thus, compression performance does not appear to be driven by direct memorization of the dataset. Notably, even for *random* texts, larger models such as Llama-3.1-8B still exhibit substantial compressing power, reliably reconstructing sequences of up to 792 tokens. This result demonstrates the impressive capacity of learnable input embeddings to control LLM generation. In particular, a single learned vector is sufficient to guide generation of nearly 800 random tokens.

A key takeaway from these results is that the model's compression ability does not depend on familiarity with specific texts or knowledge of natural language gained during pre-training. Instead, the single trainable vector itself provides language agnostic substantial capacity, allowing to store completely novel texts or random sequences of words.

4.3 Sensitivity of Compression to Text Complexity

Decoding capacity might depend on the complexity of the input text for a language model. In this section, we study how compression changes uncertainty of the model about the text.

For 50 text samples from the *PG-19* at each target length (ranging from 8 up to 1568 tokens, and to 3072 for larger models) we measured cross-entropy both before (H_{LM}) and after ($H_{\text{LM+[mem]}}$) compression (see Eq. (4)). Figure 3 compares results across Pythia and LLama models, and full results for all models are provided in Appendix D.

In Fig. 3, the models demonstrate linear relationship between cross-entropy before and after compression for not perfectly compressible texts